

Module 6 - Practical Computer Vision Exercises & Evaluation Questions

Class format: Data -> Program -> Observation -> What We Learned -> Practical Use -> Limitation -> Next Question

Recommended main dataset: Oxford-IIIT Pet for image operations and augmentation; MNIST/CIFAR-10 for quick CNN experiments.

Suggested setup: `pip install numpy pillow opencv-python matplotlib scikit-learn torch torchvision`

Exercise 1 - Inspect Image as Numbers

Program / activity

```
from PIL import Image
import numpy as np

img = Image.open("image.jpg").convert("L")
arr = np.array(img)

print(arr.shape)
print(arr[:5, :5])
print(arr.min(), arr.max())
```

Questions to evaluate understanding

1. What does each number in the matrix represent?
2. Why is a grayscale image 2-dimensional?
3. What would the shape become for an RGB image?
4. What do pixel values 0 and 255 approximately represent?
5. If image width and height both double, how does the number of pixels change?
6. Why might we convert an RGB image to grayscale?
7. Give one problem where removing color would be harmful.

Exercise 2 - Build a Vertical Edge Detector

Program / activity

```
import cv2

kernel = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])

edges = cv2.filter2D(arr, cv2.CV_32F, kernel)
```

Questions to evaluate understanding

1. What does this kernel detect?
2. Why does an edge create a large response?
3. What happens in a uniform region?
4. Why are negative values useful in the kernel?
5. What is the output of a convolution/filter called?
6. Does detecting an edge mean detecting an object? Why not?

7. Name three practical uses of edge detection.

Exercise 3 - Design a Horizontal Edge Detector

Program / activity

```
kernel = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])
```

Questions to evaluate understanding

1. What changed compared with the vertical kernel?
2. Why do different kernel values detect different features?
3. Can one image generate many different feature maps?
4. What happens if we rotate an edge kernel?
5. Why does manually designing all useful kernels become impractical?
6. What ML idea could solve that problem?

Exercise 4 - Sobel X and Sobel Y

Program / activity

```
sobel_x = np.array([
    [-1,  0,  1],
    [-2,  0,  2],
    [-1,  0,  1]
])

sobel_y = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
])

gx = cv2.filter2D(arr, cv2.CV_32F, sobel_x)
gy = cv2.filter2D(arr, cv2.CV_32F, sobel_y)
magnitude = np.sqrt(gx**2 + gy**2)
```

Questions to evaluate understanding

1. What information do gx and gy contain?
2. Why combine both?
3. What does edge magnitude represent?
4. Where will magnitude be near zero?
5. Why can Sobel be more useful than the first simple kernel?
6. Give practical uses of gradient information.

Exercise 5 - Blur with an Averaging Kernel

Program / activity

```
blur_kernel = np.ones((5, 5)) / 25
blurred = cv2.filter2D(arr, -1, blur_kernel)
```

Questions to evaluate understanding

1. What is happening to every pixel?
2. Why do fine details disappear?

3. Why should the averaging weights sum to about 1?
4. How would 9x9 blur differ from 3x3 blur?
5. Why can blur reduce noise?
6. What useful information can excessive blur destroy?
7. Why might we blur before edge detection?

Exercise 6 - Gaussian Blur + Canny

Program / activity

```
blurred = cv2.GaussianBlur(arr, (5, 5), 0)
edges = cv2.Canny(blurred, 100, 200)
```

Questions to evaluate understanding

1. Why blur before Canny?
2. What happens if the image is noisy and we skip blur?
3. What do the two thresholds roughly control?
4. What happens when thresholds are too low or too high?
5. Where could this pipeline work well?
6. Where could it fail badly?

Exercise 7 - Sharpen an Image

Program / activity

```
kernel = np.array([
    [ 0, -1,  0],
    [-1,  5, -1],
    [ 0, -1,  0]
])
sharp = cv2.filter2D(arr, -1, kernel)
```

Questions to evaluate understanding

1. Why is the center weight large and positive?
2. Why are neighbor weights negative?
3. What visual change did sharpening create?
4. Why can sharpening improve text/boundaries?
5. Why can sharpening amplify noise?
6. Name one document and one industrial use case.

Exercise 8 - Inspect RGB Channels

Program / activity

```
img_rgb = np.array(Image.open("image.jpg").convert("RGB"))
print(img_rgb.shape)
print(img_rgb[100, 100])
```

Questions to evaluate understanding

1. Why does RGB have three channels?
2. What does one RGB pixel look like numerically?
3. What color is approximately (255, 0, 0)?
4. Where is color essential?

5. Where can grayscale be enough?
6. Can differently colored objects become similar after grayscale conversion?

Exercise 9 - Convert RGB to HSV

Program / activity

```
bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
```

Questions to evaluate understanding

1. What do H, S and V represent?
2. Why can HSV make color thresholding easier than RGB?
3. Which channel mainly represents hue/color?
4. Which one mainly represents brightness?
5. Why can raw RGB thresholds fail as lighting changes?
6. Give one application where HSV may remove the need for deep learning.

Exercise 10 - Detect Green Objects with a Mask

Program / activity

```
lower = np.array([35, 50, 50])
upper = np.array([85, 255, 255])
mask = cv2.inRange(hsv, lower, upper)
```

Questions to evaluate understanding

1. What does the mask represent?
2. Why is the output mostly black and white?
3. What happens when you widen or narrow the range?
4. What lighting conditions may break this?
5. Give three practical color-threshold applications.
6. Why might a deterministic rule be preferable to a CNN here?

Exercise 11 - Horizontal Flip Augmentation

Program / activity

```
from torchvision.transforms import functional as F
flipped = F.hflip(img)
```

Questions to evaluate understanding

1. Has the image changed? Has the label changed?
2. What real-world variation does this simulate?
3. When is horizontal flip valid?
4. When is it invalid?
5. Would you flip OCR/text images? Why?
6. State the fundamental rule for a valid augmentation.

Exercise 12 - Vertical Flip

Program / activity

```
vertical = F.vflip(img)
```

Questions to evaluate understanding

1. Why is vertical flip not automatically valid?
2. Give one task where it is reasonable.
3. Give one where it creates unrealistic data.
4. What happens if training contains unrealistic augmentations?

Exercise 13 - Rotation

Program / activity

```
rotated = F.rotate(img, angle=20)
```

Questions to evaluate understanding

1. What invariance are we trying to teach?
2. Is 20 degrees valid for this task? What about 180?
3. Why does the acceptable range depend on the problem?
4. Why can rotation be dangerous for digit recognition?
5. How would you choose a range for your own project?

Exercise 14 - Translation

Program / activity

```
from torchvision.transforms import RandomAffine  
move = RandomAffine(degrees=0, translate=(0.2, 0.2))  
moved = move(img)
```

Questions to evaluate understanding

1. What changed and what stayed semantically the same?
2. What real-world condition does translation represent?
3. Should a classifier care whether the object is centered?
4. What happens if translation pushes the object mostly out of frame?

Exercise 15 - Scale

Program / activity

```
scale = RandomAffine(degrees=0, scale=(0.7, 1.3))  
scaled = scale(img)
```

Questions to evaluate understanding

1. What real-world change does scaling represent?
2. Why can the same object occupy different numbers of pixels?
3. What happens if scale variation is too aggressive?
4. Which applications need strong scale robustness?

Exercise 16 - Random Crop

Program / activity

```
from torchvision.transforms import RandomResizedCrop
crop = RandomResizedCrop(224, scale=(0.6, 1.0))
cropped = crop(img)
```

Questions to evaluate understanding

1. Why deliberately throw away pixels?
2. What real-world situation does cropping simulate?
3. How can cropping reduce reliance on exact position?
4. What happens if the discriminating feature is cropped away?
5. Why should crop policy differ for cats-vs-dogs vs breed classification?

Exercise 17 - Brightness Augmentation

Program / activity

```
from torchvision.transforms import ColorJitter
change = ColorJitter(brightness=0.5)
changed = change(img)
```

Questions to evaluate understanding

1. What real-world condition does brightness variation simulate?
2. Why should many models tolerate illumination changes?
3. When can brightness itself be useful information?
4. How would you determine a safe brightness range?

Exercise 18 - Contrast Augmentation

Program / activity

```
change = ColorJitter(contrast=0.5)
changed = change(img)
```

Questions to evaluate understanding

1. How is contrast different from brightness?
2. Why do different cameras produce different contrast?
3. What happens if contrast becomes too low?
4. Why could this help when data comes from multiple cameras?

Exercise 19 - Saturation Augmentation

Program / activity

```
change = ColorJitter(saturation=0.5)
changed = change(img)
```

Questions to evaluate understanding

1. What visual property does saturation change?
2. Why could this improve generalization?
3. Why might it damage plant-disease or fruit-ripeness classification?
4. When is color part of the label?

Exercise 20 - Hue Augmentation

Program / activity

```
change = ColorJitter(hue=0.1)
changed = change(img)
```

Questions to evaluate understanding

1. What does hue modify?
2. When might hue be irrelevant to the class?
3. Why should it be avoided for traffic lights?
4. Why is more augmentation not always better?

Exercise 21 - Blur as Augmentation

Program / activity

```
from torchvision.transforms import GaussianBlur
blur = GaussianBlur(kernel_size=7, sigma=(0.1, 3))
changed = blur(img)
```

Questions to evaluate understanding

1. How is this different from blur used as preprocessing?
2. What camera problems does random blur simulate?
3. How can it improve robustness?
4. At what point does blur destroy the label?

Exercise 22 - Perspective Distortion

Program / activity

```
from torchvision.transforms import RandomPerspective
perspective = RandomPerspective(distortion_scale=0.4, p=1)
changed = perspective(img)
```

Questions to evaluate understanding

1. What real-world effect does perspective simulate?
2. Why is it useful for documents, road signs and products?
3. When would extreme perspective become harmful?
4. Which projects need almost no perspective augmentation?

Exercise 23 - Random Erasing / Occlusion

Program / activity

```
import torchvision.transforms as T
transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.RandomErasing(p=1, scale=(0.05, 0.2))
])
changed = transform(img)
```

Questions to evaluate understanding

1. Why intentionally hide part of the image?
2. What real-world phenomenon does this simulate?
3. Give three examples of occlusion.
4. How can this reduce over-reliance on one tiny feature?

5. When would random erasing become harmful?

Exercise 24 - Combine Augmentations

Program / activity

```
augment = T.Compose([
    T.RandomResizedCrop(224),
    T.RandomHorizontalFlip(),
    T.RandomRotation(15),
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    T.RandomPerspective(distortion_scale=0.2, p=0.3)
])
```

Questions to evaluate understanding

1. Why does each run produce a different image?
2. Do all generated images make good training examples?
3. What is useful diversity vs random diversity?
4. Have we created genuinely independent new information?
5. Why does augmentation not replace collecting more real data?
6. How would you test whether augmentation improves the model?

Exercise 25 - Design an Augmentation Policy

Program / activity

```
# For your chosen task mark each as:
# YES / NO / MAYBE
# horizontal flip, vertical flip, rotation, crop, translation,
# scale, brightness, contrast, saturation, hue, blur,
# perspective, random erasing
```

Questions to evaluate understanding

1. Which augmentations did you choose and why?
2. Which did you reject and why?
3. Which transformations could corrupt labels?
4. What real-world variations are missing?
5. What parameter ranges would you use?
6. How would you validate your choices?

Exercise 26 - Create a Learnable Convolution

Program / activity

```
import torch
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=8, kernel_size=3)
print(conv.weight.shape)
```

Questions to evaluate understanding

1. What does in_channels=3 mean?
2. Why are there 8 filters?
3. Interpret [8, 3, 3, 3].
4. Who chose the actual kernel values this time?

5. How do those values change?
6. How is this different from Sobel?
7. What is the conceptual leap from classical CV to CNN?

Exercise 27 - Pass Data Through Convolution

Program / activity

```
x = torch.randn(1, 3, 32, 32)
conv = nn.Conv2d(3, 8, 3)
y = conv(x)
print(x.shape)
print(y.shape)
```

Questions to evaluate understanding

1. Why did channels go from 3 to 8?
2. Why did 32x32 become 30x30?
3. What does each output channel represent?
4. Why is the output not a class prediction yet?
5. What is a feature map?
6. How could you preserve 32x32?

Exercise 28 - Padding

Program / activity

```
conv = nn.Conv2d(3, 8, 3, padding=1)
y = conv(x)
print(y.shape)
```

Questions to evaluate understanding

1. What changed from the previous experiment?
2. What does padding add around the image?
3. Why does it preserve dimensions?
4. What happens after many convolutions without padding?
5. Why can border information matter?

Exercise 29 - Stride

Program / activity

```
conv = nn.Conv2d(3, 8, 3, stride=2, padding=1)
y = conv(x)
print(y.shape)
```

Questions to evaluate understanding

1. Why did spatial dimensions shrink?
2. What does stride mean physically?
3. What computational benefit does larger stride provide?
4. What information can it lose?
5. How is stride related to downsampling?

Exercise 30 - ReLU

Program / activity

```
relu = nn.ReLU()
x = torch.tensor([-5., -2., 0., 2., 8.])
print(relu(x))
```

Questions to evaluate understanding

1. What happened to negative values?
2. What happened to positive values?
3. Why do neural networks need nonlinear operations?
4. Why was ReLU important for deep CNNs?

Exercise 31 - Max Pooling

Program / activity

```
x = torch.tensor([[[[1.,2.,3.,4.],
                    [5.,9.,7.,8.],
                    [2.,3.,4.,1.],
                    [8.,2.,1.,6.]]]])
pool = nn.MaxPool2d(2)
print(pool(x))
```

Questions to evaluate understanding

1. What happened to the dimensions?
2. Which value is retained from each region?
3. What information is lost?
4. Why can pooling reduce computation?
5. Why do some modern architectures use strided convolutions instead?

Exercise 32 - Build a Tiny CNN

Program / activity

```
model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(32 * 8 * 8, 10)
)
```

Questions to evaluate understanding

1. Trace the shape after every layer starting from 3x32x32.
2. What do early layers tend to learn?
3. What can deeper layers learn?
4. Why do channels often increase while spatial size decreases?
5. What does Flatten do?
6. What does the final Linear layer represent?
7. How is this fundamentally different from Sobel/HOG?

Exercise 33 - Classification Output

Program / activity

```
x = torch.randn(1, 3, 32, 32)
output = model(x)
print(output)
print(output.shape)
```

Questions to evaluate understanding

1. What are the 10 output values?
2. Are they probabilities?
3. What are logits?
4. How do we turn multi-class logits into probabilities?
5. Which class is selected?
6. What loss function is commonly used for multi-class classification?

Exercise 34 - Accuracy Can Mislead

Program / activity

```
# Dataset:
# 950 normal, 50 defective
# Model predicts NORMAL for every image.
```

Questions to evaluate understanding

1. What is the accuracy?
2. Is 95% accuracy good here? Why not?
3. How many defective cases were found?
4. Which metric helps quantify missed positives?
5. What is recall? What is precision?
6. In defect detection, which error might be more costly: FP or FN? Why does business context matter?

Exercise 35 - Confusion Matrix

Program / activity

```
from sklearn.metrics import confusion_matrix
y_true = [1,1,1,0,0,0]
y_pred = [1,0,1,0,0,1]
print(confusion_matrix(y_true, y_pred))
```

Questions to evaluate understanding

1. Identify TP, FP, TN and FN.
2. Which one is a missed positive?
3. Which one is a false alarm?
4. Which affects recall?
5. Which affects precision?

Exercise 36 - Architecture Thought Experiment

Program / activity

```
# No full implementation required.
# Explain each architecture as a response to an engineering problem.
```

Questions to evaluate understanding

1. LeNet: what core idea did it establish?
2. AlexNet: why did GPU training, ReLU and large-scale data matter?
3. VGG: why repeatedly use 3x3 convolutions?
4. Inception: why use multiple kernel sizes and 1x1 convolutions?
5. ResNet: what problem appears with very deep networks?
6. Explain $y = F(x) + x$ in plain language.

Final Project Evaluation

1. What visual data does your project consume?
2. Where will the data come from?
3. How many samples are realistically available?
4. What labels exist?
5. Is the task classification, detection, segmentation, OCR, retrieval, tracking, or something else?
6. What exactly should the system output?
7. Could edge detection solve part of the problem?
8. Could color thresholds solve part of it?
9. Could contours/shapes solve part of it?
10. Can any deterministic rule avoid ML entirely?
11. Can objects rotate?
12. Can position change?
13. Can scale change?
14. Can lighting change?
15. Can color change?
16. Can perspective change?
17. Can objects be partially hidden?
18. Can image quality vary?
19. Which transformations preserve the correct label?
20. Which transformations would corrupt the label?
21. What augmentation ranges would you use?
22. Why would learned CNN features help?
23. What might early CNN layers detect?
24. What might deeper layers detect?
25. What error is most costly?
26. Is accuracy sufficient?
27. Would precision or recall matter more?
28. How will you create train/validation/test splits?
29. Is there a risk of data leakage?
30. Would simple CV or a CNN be cheaper?
31. Does inference need to be real-time?
32. Where will inference run: cloud, desktop, mobile, or edge?
33. What happens when the model sees conditions absent from training?
34. How will you detect model failure after deployment?

Rapid Oral Evaluation

1. What does a kernel actually do?
2. Why is augmentation not the same as collecting new data?
3. Name one augmentation that can destroy labels and explain why.

4. Why might a deterministic CV solution be preferable to a CNN?
5. What is the difference between a manually designed Sobel kernel and `nn.Conv2d`?

Core Teaching Principle

For every trick, students should be able to answer three things: **What changed in the data? Why is that useful?**
When will it fail? If they can answer all three, they have understood the method rather than only executed the code.